# 4-bit Array Multiplier: RTL to GDSII (SKY130)

Anshitha Penubolu (2023102014) Lavanya Patil (2024112016)

GitHub repository: `https://github.com/AnshithaP/mult4_2023102014_2024112016`

## Introduction

A 4-bit unsigned array multiplier is taken from Verilog RTL (Part A) through Yosys synthesis (Part B), OpenSTA timing (Part C) and LibreLane place and route (Part D) to a DRC- and LVS-clean SKY130 layout.

## Part A: RTL Design and Verification

Tools: Icarus Verilog 12.0 (RTL simulation), Yosys 0.70 and the SKY130 cell models in the IIC-OSIC-TOOLS 2026.10 container (synthesis and netlist simulation), GTKWave (waveform).

`rtl/full_adder.v` uses two `assign` statements: `sum = a ^ b ^ cin`, `cout = (a & b) | (cin & (a ^ b))`. `rtl/mult_array.v` follows the connection table in Section 2 of the assignment. Inputs `a`, `b` are captured in 4-bit registers `a_r`, `b_r`; 16 AND gates form $\texttt{pp}_i\texttt{[j] = a_r[j] \& b_r[i]}$. Row 0 is AND gates only: its lowest bit is `p[0]` and its upper three bits pass down. Rows 1–3 are 4-bit ripple-carry adders of four `full_adder` instances each (12 in total, written with `generate` loops). Each row's lowest sum bit is a product bit; the rest pass down, and row 3 gives `p[7:4]` into the output register `p`. All registers use the rising edge of `clk` and the asynchronous active-low `rst_n`: **16 flip-flops** (4 + 4 + 8) and a **latency of 2 clock cycles** (inputs captured at edge 1, product at edge 2). The `*` operator is not used.

`tb/tb_mult.v` makes a 10 ns clock, holds `rst_n` low for 2 cycles, applies all 256 `(a, b)` pairs on falling edges, waits the 2-cycle latency and compares `p` with `a * b` (allowed in the testbench only). It prints mismatches and one summary line, and writes `mult.vcd`.

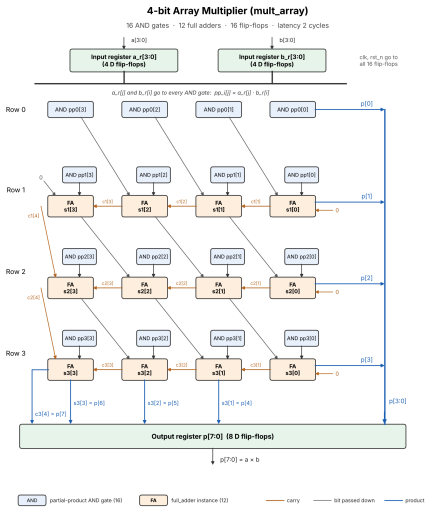



Figure 1: Block diagram: 16 AND gates, 12 full adders, registers `a_r`, `b_r`, `p` (16 flip-flops).

```
    1 0 0 1     A = 9
  x 0 1 1 1     B = 7
  -------------
    1 0 0 1     pp0 (b0 = 1)
  1 0 0 1       pp1 (b1 = 1)
1 0 0 1         pp2 (b2 = 1)
0 0 0 0         pp3 (b3 = 0)
-------------
0 0 1 1 1 1 1 1   P = 63
```

Figure 2: Hand-worked example. $A = 4 + 5 = 9$ (last digit of Anshitha Penubolu's roll number 2023102014), $B = 7$. Array rows give `p[0]`–`p[3]` = 1 and `p[7:4]` = 0011.

Table 1: RTL simulation and error test.

| Test | Result |
|---|---|
| RTL simulation | `PASS: 256 vectors checked, 0 errors` |
| Error test (`sum` changed to `a ^ b`) | `FAIL: 256 vectors checked, 95 errors` (change then undone; PASS again) |

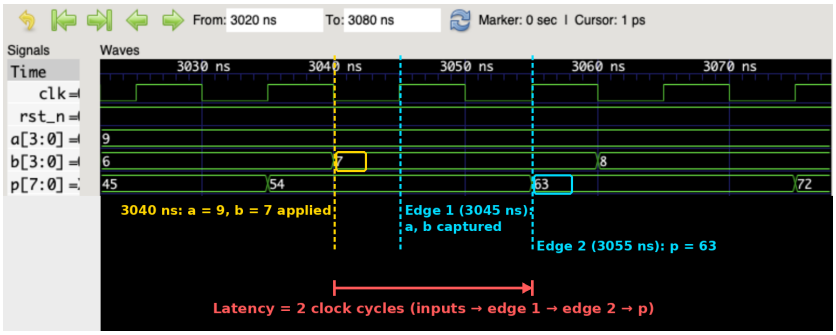



Figure 3: Waveform for $A = 9$, $B = 7$: inputs applied at 3040 ns, captured at edge 1 (3045 ns), `p` = 63 after edge 2 (3055 ns), so the latency is 2 clock cycles.

## Part B: Logic Synthesis (Yosys)

`synth/synth.tcl` reads the RTL, runs `synth -top mult_array -flatten`, maps flip-flops (`dfflibmap`) and logic (`abc`) to `sky130_fd_sc_hd` (`tt_025C_1v80`), and writes the stat report and the netlist, which passes the same testbench.

Table 2: Part B results (from `synth/mult_array_stat.rpt`).

| Metric | Value |
|---|---|
| Total number of cells | 67 |
| Number of flip-flops | 16 (`sky130_fd_sc_hd__dfrtp_1`) |
| Total cell area | 784.50 µm$^2$ |
| Flip-flop area | 400.38 µm$^2$ (16 × 25.02 µm$^2$) |
| Flip-flop area (% of total) | 51.04 % |
| 3 most-used cell types | `dfrtp_1` (16), `nand2_1` (9), `xnor2_1` (7) |
| Netlist simulation result | `PASS: 256 vectors checked, 0 errors` |

## Part C: Static Timing Analysis (OpenSTA)

Tools: IIC-OSIC-TOOLS 2026.10 container (Yosys 0.70, OpenSTA 3.1.0, LibreLane 3.1.0.dev3, OpenROAD 26Q4, Magic 8.3.685, KLayout 0.30.12, Netgen 1.5.324), PDK `sky130A`, library `sky130_fd_sc_hd`.

`sta/sta.tcl` reads the synthesized netlist with the typical-corner library `sky130_fd_sc_hd__tt_025C_1v80.lib`, puts a clock on `clk`, a 1.0 ns input delay on every input except `clk` and a 1.0 ns output delay on `p`. This is pre-layout analysis: ideal clock (no clock tree) and no extracted wire parasitics.

1. `sta sta/sta.tcl` ($T = 10$ ns): worst setup slack WNS = 7.020 ns.
2. $T_{\min} = T - \text{WNS} = 10 - 7.020 = \mathbf{2.98\ ns}$, $F_{\max} = 1000/2.98 = \mathbf{335.6\ MHz}$.
3. `STA_PERIOD=2.98 sta sta/sta.tcl` ($T = T_{\min}$): worst setup slack = 0.000 ns, confirming $T_{\min}$.

## Part C results

Table 3: Part C results (typical corner, before layout).

| Metric | Value |
|---|---|
| Worst slack at $T = 10\,\text{ns}$ | 7.020 ns |
| $T_{\min}$ | 2.98 ns |
| $F_{\max}$ | 335.6 MHz |
| Slack at $T = T_{\min}$ | 0.000 ns |
| Critical path start point | `b_r[0]` input register (instance `_106_`, `sky130_fd_sc_hd__dfrtp_1`), rising edge of `clk` |
| Critical path end point | `p[6]` output register (instance `_100_`, `sky130_fd_sc_hd__dfrtp_1`), D pin, setup check |
| Cells on the critical path | 10: launch flip-flop + 8 logic cells + capture flip-flop |
| Latency $= 2 \times T_{\min}$ | 5.96 ns |
| Throughput | 335.6 million results per second |

## Cells on the critical path

Table 4: Cells on the critical path at $T = T_{\min} = 2.98\,\text{ns}$.

| # | Instance | Cell | What it computes | Delay (ns) | Arrival (ns) |
|---|---|---|---|---|---|
| 1 | `_106_` | `dfrtp_1` | `b_r[0]` register, CLK→Q (fanout 9) | 0.411 | 0.411 |
| 2 | `_048_` | `and4_1` | $a_3a_2b_0b_1$ = `pp0[3]`·`pp1[2]`, the carry-generate of row 1, adder 2 | 0.218 | 0.629 |
| 3 | `_050_` | `a2111oi_0` | row-1 carry chain (equals `c1[3]` whenever `pp1[3]` = 0) | 0.428 | 1.057 |
| 4 | `_051_` | `a21oi_1` | row-1 carry chain (equals ¬`c1[3]` whenever `pp1[3]` = 1) | 0.094 | 1.151 |
| 5 | `_053_` | `mux2i_1` | ¬`s1[3]`: sum of row 1, adder 3 | 0.218 | 1.369 |
| 6 | `_069_` | `xnor3_1` | `s2[2]`: sum of row 2, adder 2 | 0.374 | 1.742 |
| 7 | `_074_` | `maj3_1` | `c3[2]`: carry of row 3, adder 1 | 0.371 | 2.113 |
| 8 | `_076_` | `maj3_1` | `c3[3]`: carry of row 3, adder 2 | 0.376 | 2.489 |
| 9 | `_080_` | `xor3_1` | `s3[3]` = `p[6]`: sum of row 3, adder 3 | 0.355 | 2.844 |
| 10 | `_100_` | `dfrtp_1` | `p[6]` register, setup time 0.136 ns | | required 2.844 |

The "what it computes" column of Table 4 was found by simulating every net of the netlist for all 256 values of (`a_r`, `b_r`) and matching it against the signals of the array equations in Section 2 of the assignment.

## Full adders on the critical path (task 3)

Synthesis flattened the 12 `full_adder` instances (`synth -flatten`), and ABC re-mapped the logic, so no cell is "a full adder" any more. Every one of the 8 logic cells still implements full-adder logic. Five of them produce a full-adder output directly: an XOR3/XNOR3 cell is a full-adder sum, a MAJ3 cell is a full-adder carry, and the MUX2I forms ¬`s1[3]`. The other three (`and4`, `a2111oi`, `a21oi`) are the row-1 carry chain with the partial-product AND gates folded in.

In the RTL, this is the longest chain through the array: row 1, adders $0 \to 1 \to 2 \to 3$ (carries `c1[1]`, `c1[2]`, `c1[3]`, then sum `s1[3]`); row 2, adder 2 (sum `s2[2]`); row 3, adders $1 \to 2 \to 3$ (carries `c3[2]`, `c3[3]`, then sum `s3[3]` = `p[6]`). That is **8 full adders, exactly the $2N = 8$ estimate**. Synthesis implements this chain in 8 cells. ABC computes the row-1 carry `c1[3]` directly from the partial products in three cells (`and4`, `a2111oi`, `a21oi`) instead of rippling through adders

0–2, and each of the other five full adders on the chain costs one cell (`mux2i`, `xnor3`, `maj3`, `maj3`, `xor3`). So the critical path has 8 logic cells, about one per full-adder delay. They take 2.433 ns, about 0.30 ns each. The remaining 0.547 ns of $T_{\min}$ is flip-flop overhead (0.411 ns clock-to-Q + 0.136 ns setup).

The next most critical endpoint is `p[7]` (slack 0.016 ns at $T = T_{\min}$), which takes the same path into the final carry `c3[4]`.

# Part D: Physical Design (LibreLane)

`pnr/config.json` holds the required settings: `DESIGN_NAME` `mult_array`, the two RTL files, `CLOCK_PORT` `clk`, `FP_SIZING` `absolute` and `DIE_AREA [0, 0, 150, 150]`. Everything else is a LibreLane default.

## Runs

The first `CLOCK_PERIOD` was $1.2 \times T_{\min} = 1.2 \times 2.98 = 3.576$ ns, rounded up to the next 0.5 ns = 4.0 ns. LibreLane checks setup at all 9 of its STA corners (this PDK configuration sets `TIMING_VIOLATION_CORNERS` to `*`), not only the typical one. The typical corner passed from the first run. The slow corner `max_ss_100C_1v60` (slow transistors, 100 °C, 1.60 V, maximum wire RC) failed, so the flow stopped with setup violations, and the period was raised by 0.5 ns per run as the assignment specifies (Table 5).

Table 5: Every LibreLane run. Setup slack is `timing__setup__ws` from `metrics.json`; DRC errors are Magic / KLayout.

| # | Run folder | `CLOCK_PERIOD` (ns) | Slack (ns) worst corner | Slack (ns) tt | DRC | LVS | Result |
|---|---|---|---|---|---|---|---|
| 1 | `RUN_2026-10-21_18-53-39` | 4.0 | −1.957 | +0.883 | 0 / 0 | 0 | setup fails at ss |
| 2 | `RUN_2026-10-21_18-57-48` | 4.5 | −1.457 | +1.383 | 0 / 0 | 0 | setup fails at ss |
| 3 | `RUN_2026-10-21_19-00-58` | 5.0 | −0.957 | +1.883 | 0 / 0 | 0 | setup fails at ss |
| 4 | `RUN_2026-10-21_19-04-08` | 5.5 | −0.456 | +2.382 | 0 / 0 | 0 | setup fails at ss |
| 5 | `RUN_2026-10-21_19-07-09` | 6.0 | −0.128 | +2.818 | 0 / 0 | 0 | setup fails at ss |
| 6 | `RUN_2026-10-21_19-10-12` | 6.5 | −0.157 | +3.089 | 0 / 0 | 0 | setup fails at ss |
| 7 | `RUN_2026-10-21_19-13-06` | **7.0** | **+0.076** | +3.519 | 0 / 0 | 0 | **passed (final)** |

Attempts 1 to 4 produced the same layout, and the slack improved by exactly 0.5 ns per step. Attempts 5 and 6 gained less because LibreLane re-synthesizes in every run, with the clock period as the ABC delay target (`retime -D 6000`, `-D 6500`), so the netlist changed.

## Part D results

Table 6: Part D results for the final run (`CLOCK_PERIOD` = 7.0 ns).

| Metric | Value |
|---|---|
| `CLOCK_PERIOD` used | 7.0 ns |
| Number of runs needed | 7 |
| Core area | 17,759.5 µm$^2$ (die area 22,500 µm$^2$) |
| Cells after synthesis (from Part B) | 67 (16 flip-flops + 51 logic cells) |
| Cells after place and route | 1,901 |
| Kinds of cells added | decap 1,305; fill 260; tap (`tapvpwrvgnd_1`) 245; `buf` 18; `clkdlybuf4s50` 2; `clkbuf_16` 3 |
| Wirelength | 2,340 µm (detailed route) |
| Setup slack | +0.076 ns at the worst corner (`max_ss_100C_1v60`); +3.519 ns at `nom_tt_025C_1v80` |
| Post-route $F_{\max}$ | $1000/(7.0 - 0.076) = \mathbf{144.4}$ MHz (worst corner); $1000/(7.0 - 3.519) = 287.3$ MHz at tt |
| $F_{\max}$ from Part C | 335.6 MHz (typical corner, before layout) |
| DRC errors | 0 (Magic 0, KLayout 0) |
| LVS errors | 0 |

Other signoff results: hold slack +0.139 ns (worst at `min_ff_n40C_1v95`), 0 antenna violations, 0 max-capacitance violations. There were 11 max-slew warnings at `max_ss_100C_1v60`, which LibreLane reports as warnings, not errors.

## Cells after synthesis vs. after place and route (task 4)

LibreLane runs its own synthesis, and its netlist has 52 logic gates + 16 `dfrtp` flip-flops = 68 cells, close to the 67 from Part B. The other 1,833 of the 1,901 cells were added by the physical flow (Table 7; counts from the `odb-cellfrequencytables` reports of the final run).

Table 7: Cells added by the physical flow.

| Kind | Count | Where they come from |
|---|---|---|
| `decap_3/4/6/8/12` | 1,305 | decoupling capacitors put in empty row space |
| `fill_1/2` | 260 | filler cells for the remaining gaps |
| `tapvpwrvgnd_1` | 245 | well taps, placed in a checkerboard across the rows (`FP_TAPCELL_DIST` = 13 µm) |
| `buf_2` | 17 | one buffer on each input (`a`, `b`, `rst_n`) and output (`p`) port |
| `buf_4` | 1 | setup-repair rebuffer |
| `clkdlybuf4s50_1` | 2 | fanout buffers on `rst_n`: its 16 flip-flop loads exceed the maximum fanout of 10, so one buffer drives 10 resets and the other 6 |
| `clkbuf_16` | 3 | clock tree: one root buffer and two leaf buffers, each driving 8 flip-flops |

## Layout (KLayout)

Both images were rendered with KLayout 0.30.12 from `pnr/mult_array.gds` using the SKY130 layer properties (`sky130A.lyp`).

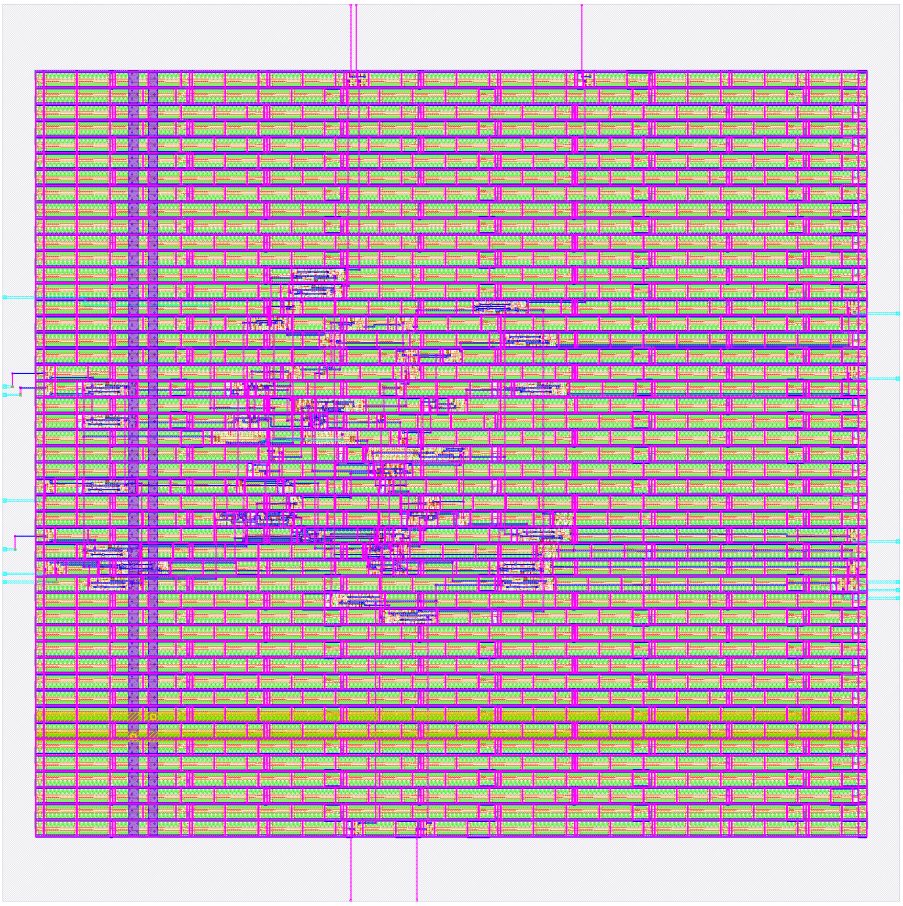


Figure 4: The whole 150 µm × 150 µm die. The core is filled with standard-cell rows. The multiplier logic sits in the middle, and every other site holds a decap, fill or tap cell. The two dark vertical stripes on the left are the met4 power straps, and the yellow band near the bottom is the met5 strap pair. The I/O pins are on the die edges.

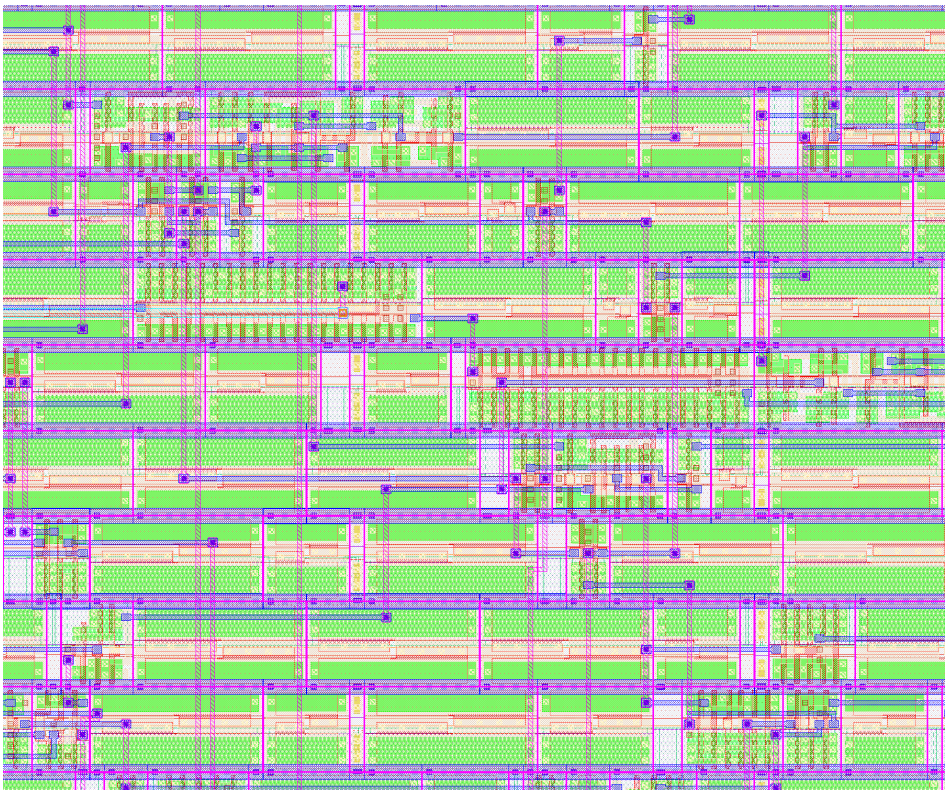


Figure 5: A 30 µm × 25 µm window over the logic. Horizontal standard-cell rows are separated by the VPWR/VGND met1 rails. Inside the cells you can see poly (red) and diffusion. Signal routing runs on met1 (blue, mostly horizontal) and met2 (magenta, vertical), with vias between layers.

## Answers to the Report Questions

**Q1. Logic cells.** Synthesis produced 67 cells, of which 16 are flip-flops, so **51 cells** implement the logic. This is not 16 + 12 = 28 because Yosys flattens the design and ABC maps the logic to whatever SKY130 gates give a small circuit, not one cell per AND gate or full adder. A full adder usually becomes several cells: an XOR/XNOR for the sum and a majority (`maj3`) or AND-OR-invert gate for the carry. ABC also merges partial-product AND gates into complex gates such as `a22oi`, `a21oi`, `a2111oi` and `o21ai`, and uses `mux2i` cells where they are smaller. It simplifies the adders with a constant 0 input (the first adder in each row has `cin` = 0, and the top adder of row 1 gets a 0 from row 0), so these act as half adders. A few inverters fix signal polarity.

**Q2. Critical path.** The critical path starts at the `b_r[0]` input register and ends at the `p[6]` output register (row 3, adder 3 sum). It has 10 cells: the launch flip-flop, 8 logic cells (`and4`, `a2111oi`, `a21oi`, `mux2i`, `xnor3`, `maj3`, `maj3`, `xor3`) and the capture flip-flop. Tracing the nets back to the RTL, the path is the 8-full-adder chain row 1 adders 0–3 → row 2 adder 2 → row 3 adders 1–3, which is exactly the estimate of about $2N = 8$ full-adder delays. After flattening, synthesis builds it from 8 cells: three compute the row-1 carry `c1[3]` straight from the partial products, and there is one cell per remaining full adder. The 8 logic cells add 2.433 ns, about 0.30 ns per full-adder delay. With 0.547 ns of flip-flop overhead, $T_{\min} = 2.98$ ns.

**Q3. Flip-flop area.** The 16 flip-flops take 400.38 µm$^2$ of the 784.50 µm$^2$ total, **51.04 %**. This is large because a flip-flop with asynchronous reset is much bigger than a basic gate: one `dfrtp_1` is 25.02 µm$^2$, while a `nand2_1` is 3.75 µm$^2$ and an `xnor2_1` is 8.76 µm$^2$. The 4-bit logic is tiny, but the design still needs 16 flip-flops to register both inputs and the output. As the multiplier gets wider, the logic grows with $N^2$ while the flip-flops grow only with $4N$, so this share falls for larger multipliers.

**Q4. Post-route $F_{max}$ and added cells.** Post-route $F_{max}$ is $1000/(7.0 - 0.076) = 144.4$ MHz at the worst corner, and $1000/(7.0 - 3.519) = 287.3$ MHz at the typical corner, against 335.6 MHz in Part C. Part C was an optimistic pre-layout check: one corner (tt), an ideal clock, no wires and fixed 1 ns I/O delays. LibreLane's signoff STA is stricter in five ways:

- it includes the extracted RC of the 2,340 µm of routed wire, which slows edges and loads every driver;
- the clock goes through a real tree (about 0.32 ns insertion delay at tt), with 0.25 ns of clock uncertainty and 5 % derating on top;
- inputs and outputs see real conditions (an `inv_2` driving each input, 33 fF on each output, I/O delays of 20 % of the period);
- the netlist is different, because LibreLane re-synthesizes with the clock period as ABC's delay target and at 7 ns picks smaller, slower gates (its critical path runs `b_r[1]` register → `o2111ai` → `o31ai` → `nand3`/`nor2b` → `a21oi` → `xnor3` → three `maj3` → `p[7]` register);
- above all, setup must pass at all nine corners. At `max_ss_100C_1v60` that path arrives at 6.93 ns instead of 3.42 ns at tt, about 2× slower, which is why the period had to rise to 7.0 ns.

The flow added five kinds of cells:

- **buffers** (18 `buf`, 2 `clkdlybuf4s50`): give every I/O port a strong, known driver, repair setup, and split the 16-load `rst_n` net to respect the maximum fanout;
- **clock buffers** (3 `clkbuf_16`): form the clock tree that delivers the clock to all 16 flip-flops with low skew;
- **tap cells** (245): tie the n-well to VPWR and the substrate to VGND at regular intervals, so the parasitic thyristor cannot latch up;
- **decap cells** (1,305): add on-chip capacitance between VPWR and VGND that supplies the switching current and keeps supply noise and IR drop low;
- **fill cells** (260): close the remaining gaps so the wells, implants and power rails stay continuous along every row, as manufacturing requires.

**Q5. DRC and LVS.** DRC (design rule check) verifies that every shape in the layout obeys the foundry's geometric rules for its layer: minimum width, spacing, enclosure, overlap, area and density (for example, SKY130 met1 needs 0.14 µm minimum width and spacing). LVS (layout versus schematic) extracts the devices and their connections from the layout and checks that they form exactly the circuit in the netlist that was placed and routed: no shorts, no opens, and no missing or extra devices or pins. Both must pass because they catch different failures. A DRC violation means the masks may not print reliably (wires short or break, so yield drops). An LVS mismatch means the chip that gets built is not the circuit we simulated and timed. A layout can be DRC-clean and still be the wrong circuit, or be the right circuit and still be unmanufacturable. Our final run has 0 DRC errors in both Magic and KLayout, and Netgen LVS reports 0 errors (no device, net or pin mismatches).

**Q6. Scaling to 8 bits.** From Section 2 of the assignment, an $N$-bit array uses $N^2$ AND gates, $N(N-1)$ full adders and has a longest path of about $2N$ full-adder delays (Table 8).

Table 8: Scaling the array multiplier from $N = 4$ to $N = 8$.

| | $N = 4$ | $N = 8$ | Ratio |
|---|---|---|---|
| AND gates, $N^2$ | 16 | 64 | 4× |
| Full adders, $N(N-1)$ | 12 | $8 \times 7 = 56$ | 4.7× |
| Flip-flops ($2N$ in + $2N$ out) | 16 | 32 | 2× |
| Critical path, about $2N$ FA delays | 8 | 16 | 2× |

Delay estimate from our Part C numbers: the 4-bit array logic takes $2.844 - 0.411 = 2.433$ ns for about 8 full-adder delays, so one full-adder delay is about 0.30 ns. The flip-flop overhead (clock-to-Q + setup $= 0.411 + 0.136 = 0.547$ ns) does not change with $N$. For $N = 8$:

$$T_{\min} \approx 0.547 + 16 \times 0.304 = 0.547 + 4.866 \approx 5.41 \text{ ns}, \qquad F_{\max} \approx 185 \text{ MHz},$$

compared with 2.98 ns and 335.6 MHz for $N = 4$. The number of full adders grows about 4.7 times (area grows as $N^2$), while the critical-path delay roughly doubles (delay grows linearly with $N$, about 1.8× here because the flip-flop overhead is fixed).

## Conclusion

The multiplier passes all 256 input combinations in RTL and gate-level simulation. Synthesis gives 67 cells (784.50 µm$^2$, 51.04 % flip-flops). Pre-layout STA gives $T_{\min} = 2.98$ ns ($F_{\max} = 335.6$ MHz) along an 8-full-adder critical path, matching the $2N$ estimate. Place and route closes at a 7.0 ns clock across all nine corners (144.4 MHz worst corner) with 0 DRC and 0 LVS errors.

## Individual Contributions

**Anshitha Penubolu (2023102014):** Parts A and B (RTL, testbench, simulation, error test, worked example, waveform, block diagram, synthesis and netlist simulation), their report sections, and Q1, Q3 and Q6.
**Lavanya Patil (2024112016):** Parts C and D (static timing analysis, critical-path study, LibreLane runs, layout and signoff), their report sections, and Q2, Q4 and Q5.

AI tools helped write parts of the code.